

versão corrigida de cada código

ChatGPT

```
# =====
```

```
for dia in range(1, DIAS + 1):
```

```
    print(f"\n----- DIA {dia} -----")
```

```
        for i in range(len(bairros)):
```

```
            # Variação aleatória da demanda
```

```
            variacao = random.uniform(-0.2, 0.2)
```

```
            demanda_dia = demandas[i] * (1 + variacao)
```

```
            # Atualiza volume
```

```
            volumes_simulados[i] -= demanda_dia
```

```
            # Impede valores negativos
```

```
            if volumes_simulados[i] < 0:
```

```
                volumes_simulados[i] = 0
```

```
            # Armazena histórico
```

```
            historico[bairros[i]].append(volumes_simulados[i])
```

```
            # Evita divisão por zero
```

```

if demanda_dia > 0:

    autonomia_dia = volumes_simulados[i] / demanda_dia

else:

    autonomia_dia = 0


risco_dia = classificar_risco(autonomia_dia)


print(f"\nBairro: {bairros[i]}")

print(f"Demanda do dia: {demanda_dia:.2f} m³")

print(f"Volume restante: {volumes_simulados[i]:.2f} m³")

print(f"Autonomia: {autonomia_dia:.2f} dias")

print(f"Risco: {risco_dia}")


# Verifica situação crítica

if risco_dia == "Alto":

    dias_criticos.append(

        f"{bairros[i]} - Dia {dia}"

    )


# =====

# EXIBIÇÃO DOS DIAS CRÍTICOS

# =====


print("\n=====")

print("DIAS EM SITUAÇÃO CRÍTICA")

print("=====")

```

```
if len(dias_criticos) == 0:
```

```
    print("Nenhum bairro entrou em situação crítica.")
```

```
else:
```

```
    for item in dias_criticos:
```

```
        print(item)
```

```
# =====
```

```
# GRÁFICO FINAL - EVOLUÇÃO DOS RESERVATÓRIOS
```

```
# =====
```

```
plt.figure(figsize=(12, 6))
```

```
for bairro in bairros:
```

```
    plt.plot(
```

```
        range(1, DIAS + 1),
```

```
        historico[bairro],
```

```
        marker='o',
```

```
        label=bairro
```

```
    )
```

```
plt.xlabel("Dias")
```

```
plt.ylabel("Volume Disponível (m³)")
```

```
plt.title("Evolução do Volume dos Reservatórios")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Gemini

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import random
```

```
# Configuração para evitar avisos de gráficos no VS Code
```

```
plt.rcParams['figure.facecolor'] = 'white'
```

```
def calcular_risco(autonomia):
```

```
    """Classifica o risco de desabastecimento."""
```

```
    if autonomia > 3:
```

```
        return "Baixo"
```

```
    elif 1 <= autonomia <= 3:
```

```
        return "Médio"
```

```
    else:
```

```
        return "Alto"
```

```
def simular_7_dias(bairros_original):  
    """  
    Simula o sistema por 7 dias.  
    Correção: Adicionado controle para volume não ficar negativo.  
    """  
    historico_volumes = {b['nome']: [b['volume_atual']] for b in bairros_original}  
    eventos_criticos = []  
  
    # Cópia profunda dos dados para a simulação  
    bairros_simu = [dict(b) for b in bairros_original]  
  
    for dia in range(1, 8):  
        for b in bairros_simu:  
            # Variação aleatória da demanda (Desafio Adicional)  
            variacao = random.uniform(0.8, 1.2)  
            demanda_do_dia = b['demanda'] * variacao  
  
            # Cálculo de perda e necessidade total de saída do reservatório  
            perda_no_dia = demanda_do_dia * (b['perda_pct'] / 100)  
            total_saida = demanda_do_dia + perda_no_dia  
  
            # Atualização do volume (Não permite saldo negativo)  
            if b['volume_atual'] >= total_saida:  
                b['volume_atual'] -= total_saida  
            else:
```

```
b['volume_atual'] = 0
```

```
eventos_criticos.append(f"Dia {dia}: {b['nome']} ESGOTOU o volume!")
```

```
# Armazena histórico
```

```
historico_volumes[b['nome']].append(b['volume_atual'])
```

```
return historico_volumes, eventos_criticos
```

```
def gerar_relatorio_e_graficos():
```

```
# 1. Definição dos Dados (4 Bairros Conforme Solicitado)
```

```
# Estrutura: Nome, Demanda, Capacidade, Volume Inicial, % Perda, Custo/m3
```

```
dados_bairros = [
```

```
    {'nome': 'Centro', 'demanda': 500, 'capacidade': 2500, 'volume_atual': 2200, 'perda_pct':  
12, 'custo_m3': 2.10},
```

```
    {'nome': 'Zona Norte', 'demanda': 800, 'capacidade': 2000, 'volume_atual': 900,  
'perda_pct': 18, 'custo_m3': 2.50},
```

```
    {'nome': 'Industrial', 'demanda': 1500, 'capacidade': 6000, 'volume_atual': 5500,  
'perda_pct': 5, 'custo_m3': 3.00},
```

```
    {'nome': 'Parque Sul', 'demanda': 200, 'capacidade': 500, 'volume_atual': 150,  
'perda_pct': 25, 'custo_m3': 1.80}
```

```
]
```

```
# 2. Cálculos Iniciais (Tabela)
```

```
lista_processada = []
```

```
for b in dados_bairros:
```

```
    perda_m3 = b['demanda'] * (b['perda_pct'] / 100)
```

```
    vol_entregue = b['demanda']
```

```
    custo_total = (vol_entregue + perda_m3) * b['custo_m3']
```

```
# Correção: Evitar divisão por zero se a demanda for 0
```

```
autonomia = b['volume_atual'] / b['demanda'] if b['demanda'] > 0 else 99
```

```
lista_processada.append({
```

```
    'Bairro': b['nome'],
```

```
    'Entrega (m³)': vol_entregue,
```

```
    'Perda (m³)': round(perda_m3, 2),
```

```
    'Custo Diário (R$)': round(custo_total, 2),
```

```
    'Autonomia (Dias)': round(autonomia, 2),
```

```
    'Risco': calcular_risco(autonomia)
```

```
})
```

```
df = pd.DataFrame(lista_processada)
```

```
# 3. Execução da Simulação de 7 dias
```

```
hist_volumes, alertas = simular_7_dias(dados_bairros)
```

```
# --- APRESENTAÇÃO DOS RESULTADOS ---
```

```
print("\n" + "="*60)
```

```
print("    SISTEMA DE ANÁLISE DE ESTABILIDADE HÍDRICA")
```

```
print("="*60)
```

```
print(df.to_string(index=False))
```

```
# Identificação do bairro crítico
```

```
critico = df.loc[df['Autonomia (Dias)'].idxmin()]
```

```
print("-" * 60)
```

```
print(f"BAIRRO MAIS CRÍTICO: {critico['Bairro']} (Risco: {critico['Risco']})")
```

```
print("-" * 60)
```

```
if alertas:
```

```
    print("\nOCORRÊNCIAS NA SIMULAÇÃO (7 DIAS):")
```

```
    for a in set(alertas): # set() para não repetir alertas iguais
```

```
        print(f"⚠️ {a}")
```

```
# --- GERAÇÃO DE GRÁFICOS ---
```

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

```
plt.subplots_adjust(hspace=0.4)
```

```
# G1: Demanda vs Entregue
```

```
df.plot(x='Bairro', y=['Entrega (m³)', 'Perda (m³)'], kind='bar', ax=axes[0, 0], color=['teal', 'tomato'])
```

```
axes[0, 0].set_title("Volume de Entrega vs Perda")
```

```
# G2: Custo
```

```
df.plot(x='Bairro', y='Custo Diário (R$)', kind='bar', ax=axes[0, 1], color='goldenrod')
```

```
axes[0, 1].set_title("Custo Operacional por Bairro")
```

```
# G3: Evolução dos Reservatórios (Linha)
```

```
for bairro, volumes in hist_volumes.items():
```

```
    axes[1, 0].plot(range(8), volumes, marker='o', label=bairro)
```

```
axes[1, 0].set_title("Simulação: Evolução do Volume em 7 Dias")
```

```
axes[1, 0].set_xlabel("Dia")
```



```
axes[1, 0].legend()
```

```
# G4: Riscos
```

```
df['Risco'].value_counts().plot(kind='pie', ax=axes[1, 1], autopct='%1.1f%%', colors=['red',  
'orange', 'green'])
```

```
axes[1, 1].set_title("Distribuição de Riscos")
```

```
plt.show()
```

```
# Executar o programa
```

```
if __name__ == "__main__":
```

```
    gerar_relatorio_e_graficos()
```

Copilot

```
# Identificação do bairro mais crítico (robusto contra NaN)
```

```
bairro_critico = df.loc[df["Autonomia (dias)"].fillna(float("inf")).idxmin(), "Bairro"]
```

```
print(f"\nBairro mais crítico: {bairro_critico}")
```

```
# Gráfico de barras com deslocamento para melhor visualização
```

```
import numpy as np
```

```
x = np.arange(len(df["Bairro"]))
```

```
plt.figure(figsize=(10,6))
```

```
plt.bar(x-0.2, df["Demanda (m³)"], width=0.4, label="Demanda")
```

```
plt.bar(x+0.2, df["Volume Entregue (m³)"], width=0.4, label="Volume Entregue")

plt.xticks(x, df["Bairro"])

plt.title("Comparação Demanda vs Volume Entregue")

plt.legend()

plt.show()
```

```
# Gráfico de pizza com cores dinâmicas

cores = {"Baixo": "green", "Médio": "yellow", "Alto": "red"}

df["Risco"].value_counts().plot(kind="pie", autopct="%1.1f%%",

                                colors=[cores[r] for r in df["Risco"].value_counts().index])

plt.title("Distribuição dos Níveis de Risco")

plt.ylabel("")

plt.show()
```